

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO SEMINAR CUỐI KÌ

Đề tài: A Triple of Vector Stores: Qdrant, Weaviate, and Milvus

Môn học: Nhập môn Dữ liệu lớn

Sinh viên thực hiện:

Lê Xuân Trí - 23120099

Nguyễn Hồ Anh Tuấn - 23120185

Trần Hữu Kim Thành - 23120166

Trần Lê Trung Trực - 23120180

Giáo viên hướng dẫn:

Thầy Trần Huy Bân

Ngày 7 tháng 6 năm 2026

Mục lục

1	Giới thiệu	1
1.1	Vector Database và vai trò trong Big Data	1
1.2	Vai trò trong RAG, Semantic Search và AI Systems	1
1.3	Mục tiêu báo cáo	2
2	Tổng quan ba hệ quản trị Vector Database	2
2.1	Qdrant	2
2.2	Weaviate	3
2.3	Milvus	3
2.4	Mức độ phổ biến trong cộng đồng	3
3	Mã nguồn mở và chính sách giá	4
4	Phân tích kiến trúc	5
4.1	Qdrant: gọn nhẹ cho filtered retrieval	5
4.2	Weaviate: hybrid retrieval trong cùng query engine	6
4.3	Milvus: phân tán để phục vụ scale lớn	6
4.4	So sánh kiến trúc	7
5	Đánh giá so sánh	8
5.1	Thiết lập và phương pháp	8
5.2	Kết quả snapshot	8
5.3	Recall-latency tradeoff và filter	9
5.4	Tài nguyên và Developer Experience	10
5.5	Hạn chế	10
6	Thảo luận	10
7	Kết luận	12
	Tài liệu tham khảo	14

1 Giới thiệu

Trong những năm gần đây, các mô hình ngôn ngữ lớn (Large Language Model – LLM) như GPT, Gemini hay Qwen đã chứng minh khả năng sinh ngôn ngữ tự nhiên vượt trội. Tuy nhiên, LLM vẫn tồn tại ba giới hạn nghiêm trọng: (i) kiến thức bị đóng băng tại thời điểm huấn luyện (*knowledge cutoff*), (ii) có thể sinh ra thông tin không chính xác một cách tự tin (*hallucination*), và (iii) không thể truy cập dữ liệu nội bộ của tổ chức [5]. Retrieval-Augmented Generation (RAG) là kiến trúc được đề xuất nhằm khắc phục cả ba giới hạn trên bằng cách bổ sung bước truy hồi ngữ cảnh (*retrieval*) trước khi mô hình sinh câu trả lời.

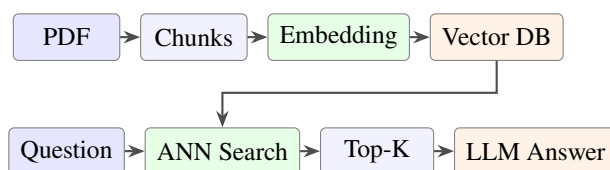
1.1 Vector Database và vai trò trong Big Data

Trong pipeline RAG, tài liệu được chia thành các đoạn nhỏ (*chunk*), mỗi đoạn được chuyển đổi thành vector embedding — một mảng số thực nhiều chiều (thường 768–1536 chiều) biểu diễn ngữ nghĩa của đoạn văn bản. Các vector này được lưu trữ trong **Vector Database** — một hệ quản trị cơ sở dữ liệu chuyên biệt cho việc lưu trữ và tìm kiếm vector nhiều chiều dựa trên độ tương đồng ngữ nghĩa.

Khác với cơ sở dữ liệu truyền thống (SQL, NoSQL) vốn tối ưu cho truy vấn chính xác (WHERE, LIKE), Vector Database giải quyết bài toán *tìm kiếm theo ngữ nghĩa*: thay vì so khớp từ khóa, hệ thống tìm các chunk có ý nghĩa gần nhất với câu hỏi của người dùng. Hầu hết Vector Database hiện đại sử dụng thuật toán HNSW (Hierarchical Navigable Small World) [6] — một thuật toán tìm kiếm láng giềng gần đúng (Approximate Nearest Neighbor – ANN) dạng đồ thị đa tầng, cho phép truy vấn với độ trễ dưới 10 ms trên tập dữ liệu hàng triệu vector [1].

1.2 Vai trò trong RAG, Semantic Search và AI Systems

Pipeline RAG hoạt động theo hai pha. **Pha indexing**: tài liệu PDF được cắt thành chunk, embedding thành vector, và lưu vào Vector Database kèm metadata. **Pha answering**: câu hỏi của người dùng được embedding thành query vector, Vector Database tìm Top-K chunk gần nghĩa nhất, các chunk này được đưa vào prompt để LLM sinh câu trả lời có ngữ cảnh. Nếu tầng retrieval trả về context sai, LLM sẽ không có bằng chứng đáng tin cậy để trả lời chính xác dù bản thân mô hình rất mạnh.



Hình 1: Pipeline RAG: pha indexing (trên) và pha answering (dưới).

1.3 Mục tiêu báo cáo

Báo cáo này phân tích ba hệ quản trị Vector Database tiêu biểu — **Qdrant**, **Weaviate** và **Milvus** — từ góc nhìn kiến trúc, khả năng truy hồi, chi phí vận hành và tiêu chí lựa chọn cho production. Nhóm không chỉ tổng hợp lý thuyết mà còn xây dựng hệ thống benchmark full-stack để đánh giá bằng số liệu thực nghiệm.

2 Tổng quan ba hệ quản trị Vector Database

Qdrant, Weaviate và Milvus đều thuộc nhóm Vector Database: hệ quản trị dữ liệu được thiết kế để lưu trữ embedding, lập chỉ mục vector nhiều chiều và trả về các điểm dữ liệu gần nhất theo độ đo như cosine similarity, dot product hoặc Euclidean distance. Trong hệ thống Big Data hiện đại, lớp này thường đứng giữa data lake/document store và ứng dụng AI, đặc biệt trong Retrieval-Augmented Generation (RAG), semantic search, recommendation và duplicate detection. Điểm quan trọng là ba công cụ không tối ưu cho cùng một ràng buộc. Qdrant nhấn mạnh filtered vector search gọn nhẹ; Weaviate nhấn mạnh trải nghiệm xây dựng ứng dụng AI/hybrid RAG; Milvus nhấn mạnh scale phân tán và tách vai trò compute/storage.

2.1 Qdrant

Qdrant là Vector Database viết bằng **Rust**, ra đời năm 2021 bởi Qdrant Solutions GmbH. Triết lý thiết kế của Qdrant là *performance-first*: giảm overhead runtime, giữ footprint gọn và tối ưu truy vấn có payload filter [12]. Trong mô hình dữ liệu của Qdrant, vector đi kèm payload dạng key-value; payload này có thể được lập chỉ mục để hỗ trợ các truy vấn như “tìm đoạn văn gần nhất nhưng chỉ trong tài liệu của phòng ban X”. Vì vậy, Qdrant phù hợp với knowledge base hoặc RAG nhiều tenant, nơi mỗi truy vấn thường cần lọc theo quyền truy cập, loại tài liệu, domain hoặc thời

gian.

2.2 Weaviate

Weaviate là Vector Database viết bằng **Go**, ra đời năm 2019 bởi Weaviate B.V. Điểm khác biệt cốt lõi là định hướng *AI-native*: ngoài vector HNSW index, Weaviate còn duy trì inverted index cho keyword search và cung cấp module tích hợp vectorizer, reranker, generative model [18, 20]. Weaviate vì thế không chỉ là một ANN engine, mà còn là một retrieval platform có schema object, hybrid score và GraphQL/REST API. Công cụ này phù hợp khi RAG cần kết hợp semantic search với keyword chính xác như tên riêng, mã sản phẩm, thuật ngữ chuyên ngành hoặc các truy vấn có phần “literal match” quan trọng.

2.3 Milvus

Milvus là Vector Database sử dụng **C++** cho lõi tính toán và **Go** cho các service điều phối, ra đời năm 2019 và là *graduated project* của LF AI & Data Foundation [17]. Milvus theo hướng *distributed-first*: tách compute khỏi storage, chia vai trò thành Proxy, Coordinator, QueryNode, DataNode và IndexNode. Thay vì tối ưu cho trải nghiệm một container, Milvus tối ưu cho hệ thống enterprise có nhiều collection, corpus rất lớn, yêu cầu HA hoặc cần tận dụng nhiều loại index như IVF, HNSW, DiskANN/GPU thông qua Knowhere [10].

2.4 Mức độ phổ biến trong cộng đồng

GitHub stars, số lượng issue/PR, connector trong ecosystem AI và DB-Engines ranking chỉ nên xem là tín hiệu cộng đồng, không phải bằng chứng trực tiếp về chất lượng kỹ thuật. Tại thời điểm nhóm tổng hợp tài liệu cho seminar, Milvus có cộng đồng GitHub lớn nhất trong ba công cụ và xuất hiện sớm trong nhiều benchmark ANN/vector database; Qdrant tăng trưởng nhanh nhờ làn sóng RAG/AI agents và cách triển khai gọn; Weaviate nổi bật trong nhóm developer xây dựng hybrid RAG và ứng dụng có schema object rõ ràng [3]. Vì vậy, trong bối cảnh Big Data, ba công cụ đều phổ biến nhưng phổ biến ở các phân khúc khác nhau: Milvus thiên về enterprise/scale, Qdrant thiên về production RAG gọn nhẹ, Weaviate thiên về developer-facing AI application.

Bảng 1: Tổng quan thiết kế và phân khúc sử dụng của ba Vector Database.

Tiêu chí	Qdrant	Weaviate	Milvus
Năm ra đời	2021	2019	2019
Ngôn ngữ core	Rust	Go	C++ & Go
Trọng tâm thiết kế	Filtered vector search	Hybrid RAG platform	Enterprise vector scale
Đối tượng phù hợp	Team cần latency/footprint gọn	Ứng dụng cần semantic + keyword	Hệ thống lớn cần HA/scale
Kỹ thuật nổi bật	Payload index, query planner	HNSW + inverted index + modules	Disaggregated services, Knowhere
Tín hiệu cộng đồng	Tăng trưởng nhanh trong RAG	Mạnh ở AI app/hybrid search	Cộng đồng lớn, LF AI & Data

3 Mã nguồn mở và chính sách giá

Cả ba công cụ đều là mã nguồn mở với license permissive, cho phép sử dụng thương mại, chỉnh sửa và triển khai self-host mà không phải trả phí bản quyền. Vì vậy, câu hỏi “công cụ nào miễn phí” không đủ để ra quyết định production. Chi phí thực tế thường đến từ RAM, CPU/GPU, storage, backup, monitoring, network, SLA và năng lực vận hành. Bảng 2 tóm tắt license, mô hình triển khai và free tier cloud tại thời điểm nhóm hoàn thiện báo cáo.

Bảng 2: So sánh license, self-host và chính sách cloud/free tier.

Tiêu chí	Qdrant	Weaviate	Milvus
License	Apache 2.0	BSD 3-Clause	Apache 2.0
Self-host	Docker/Kubernetes	Docker/Kubernetes/Helm	Docker/Kubernetes/Operator
Managed cloud	Qdrant Cloud	Weaviate Cloud	Zilliz Cloud
Free tier/trial	1 node, 1GB RAM, 4GB disk [14]	Always free: 100,000 objects, 1GB memory, 10GB disk [21]	5GB storage, khoảng 1M vectors [22]
Governance	Company-led	Company-led	LF AI & Data Foundation
Hàm ý chi phí	Ops nhẹ khi scale vừa	Trả phí cho cloud/DX/module	Hạ tầng lớn hơn nhưng scale tốt

Khác biệt đáng chú ý nằm ở governance. Milvus là *graduated project* của LF AI & Data Foundation, nên hình ảnh cộng đồng của Milvus gắn với một foundation độc lập hơn là một vendor duy nhất. Qdrant và Weaviate vẫn theo mô hình company-led: mã nguồn mở, nhưng roadmap và cloud service chủ yếu do công ty sáng lập điều phối. Trong bối cảnh học thuật, khác biệt này không làm công cụ nào “mở” hơn theo nghĩa license, nhưng ảnh hưởng đến cách cộng đồng đóng góp, mức độ phụ thuộc vendor và chiến lược support dài hạn.

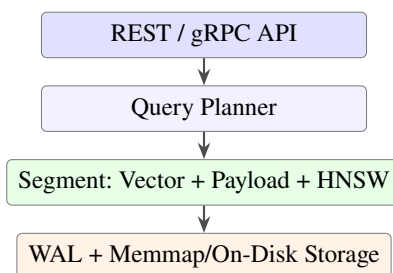
Với nhóm triển khai thực tế, self-host là lựa chọn hợp lý để học kiến trúc và benchmark vì có thể kiểm soát version, cấu hình, tài nguyên và dữ liệu. Managed cloud phù hợp hơn khi yêu cầu chính là giảm thời gian vận hành hoặc cần SLA. Free tier của ba nền tảng nên được xem là môi trường prototype; khi dữ liệu tăng lên hàng triệu đến hàng trăm triệu vector, chi phí chính sẽ chuyển sang bộ nhớ, storage, replication, backup và network egress.

4 Phân tích kiến trúc

Ba hệ thống cùng dùng ANN index để tìm vector gần nhất, nhưng khác nhau ở đường đi dữ liệu và ranh giới vận hành. Qdrant tối ưu sự gọn nhẹ và filtered search; Weaviate đặt hybrid retrieval vào query engine; Milvus tách rời compute/storage để mở rộng theo vai trò.

4.1 Qdrant: gọn nhẹ cho filtered retrieval

Qdrant được thiết kế theo hướng tối giản vận hành: một binary Rust, ít dependency, hỗ trợ REST và gRPC. Đơn vị lưu trữ chính là **segment**; mỗi segment giữ vector storage, payload storage và HNSW index riêng. Write-Ahead Log bảo vệ write path, còn storage có thể dùng RAM hoặc memory-mapped files để tận dụng OS cache [12, 13].



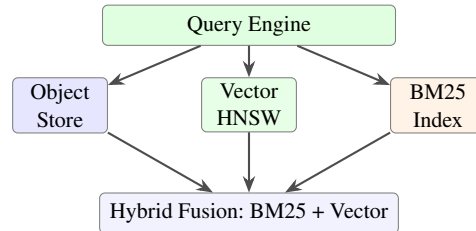
Hình 2: Kiến trúc Qdrant xoay quanh segment và payload-aware query path.

Kỹ thuật đáng chú ý nhất của Qdrant là tối ưu **filtered vector search**. Trong RAG production, truy vấn thường kèm điều kiện metadata như tenant, ACL, loại tài liệu hoặc timestamp. Nếu filter được xử lý quá sớm, graph HNSW có thể bị thừa; nếu filter hậu xử lý, Top-K hợp lệ có thể thiếu. Qdrant giải quyết bằng payload index, query planner và filter-aware search path để chọn chiến lược phù hợp theo độ chọn lọc của filter [11]. Ngoài ra, Qdrant hỗ trợ scalar/product/binary quantization, giúp giảm bộ nhớ hoặc tăng tốc so sánh khi chấp nhận đánh đổi một phần độ chính xác [4].

Trade-off của Qdrant là rõ ràng: triển khai gọn, latency thấp và footprint nhỏ trong benchmark của nhóm, nhưng không có BM25 native như Weaviate và không đặt trọng tâm vào kiến trúc phân tán nhiều thành phần như Milvus. Vì vậy Qdrant phù hợp nhất khi bài toán cần filtered retrieval, ít ops và thời gian tích hợp nhanh.

4.2 Weaviate: hybrid retrieval trong cùng query engine

Weaviate được thiết kế như một nền tảng vector database *AI-native*. Trong mỗi shard, Weaviate quản lý đồng thời object store, vector HNSW index và inverted index cho BM25/property filtering [18]. Cách đặt lexical index và vector index cạnh nhau cho phép Weaviate phục vụ native Hybrid Search thay vì phải ghép thêm search engine bên ngoài.



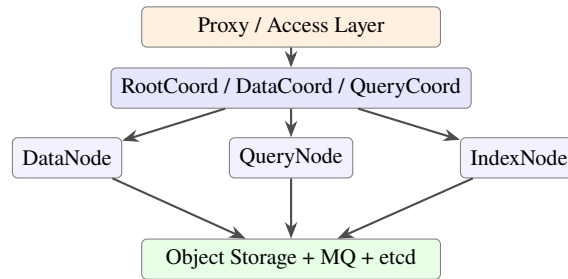
Hình 3: Weaviate kết hợp object store, HNSW và BM25 trong cùng shard.

Hybrid Search hữu ích vì vector search mạnh với paraphrase nhưng có thể bỏ sót mã lỗi, tên riêng hoặc ký hiệu; BM25 lại mạnh với keyword chính xác nhưng không hiểu đồng nghĩa [15]. Weaviate dùng tham số α để cân bằng hai tín hiệu và có thể hợp nhất ranking bằng các kỹ thuật như Reciprocal Rank Fusion [19, 2]. Hệ module như vectorizer, reranker và generative integration cũng giúp team RAG dựng pipeline nhanh hơn [20].

Trade-off của Weaviate là hy sinh một phần độ trễ dense-only để đổi lấy hybrid search và trải nghiệm phát triển thuận tiện hơn. Trong snapshot benchmark của nhóm, Weaviate không nhanh nhất khi chỉ đo dense retrieval, nhưng lợi thế thiết kế của nó nằm ở trường hợp cần keyword evidence và semantic similarity trong cùng API.

4.3 Milvus: phân tán để phục vụ scale lớn

Milvus chọn kiến trúc *disaggregated*: tách access, coordinator, worker và storage layer [17, 9]. Proxy nhận request; RootCoord/DataCoord/QueryCoord điều phối metadata, segment và load balancing; DataNode xử lý ingest, QueryNode phục vụ search, IndexNode build index; storage layer dùng object storage, etcd và message queue. Thiết kế này làm deployment phức tạp hơn, nhưng cho phép scale từng vai trò độc lập.



Hình 4: Milvus tách compute khỏi storage để scale theo từng vai trò.

Bên dưới Milvus là **Knowhere**, lớp trừu tượng bao bọc Faiss, HNSWlib, DiskANN và đường GPU khi có CUDA [8, 7, 16]. Vòng đời dữ liệu cũng cần chú ý khi benchmark: dữ liệu đi qua insert, flush, sealed segment, build index, load, rồi mới search tối ưu [10]. Nếu gộp các bước này vào một metric duy nhất, latency có thể bị diễn giải sai.

Trade-off của Milvus là scale và HA đổi lấy ops complexity. Milvus phù hợp khi corpus lớn, cần replica, GPU hoặc mở rộng từng vai trò; ngược lại, với prototype nhỏ, nhiều dependency như etcd, MinIO và message queue có thể là chi phí vận hành không cần thiết.

4.4 So sánh kiến trúc

Bảng 3: So sánh kiến trúc và trade-off chính.

Tiêu chí	Qdrant	Weaviate	Milvus
Mô hình	Single binary	Single database service	Distributed services
Tối ưu chính	Filtered vector search	Hybrid retrieval	Enterprise scale
Index nổi bật	HNSW + payload index	HNSW + BM25 index	Knowhere: Faiss/HNSWlib/DiskANN
Keyword/hybrid	Sparse vectors, external logic	BM25 native + vector	Dense/sparse, tùy cấu hình
Schema	Payload linh hoạt	Schema/module oriented	FieldSchema chặt
Vận hành	Gọn, 1 container	Gọn, nhiều module tùy chọn	Phức tạp hơn, 3+ container
Điểm mạnh	Latency/filter/DX	Hybrid RAG ergonomics	HA, GPU, scale per-role
Đánh đổi	Ít native lexical search	Dense-only overhead cao hơn	Ops complexity cao

Tóm lại, Qdrant phù hợp khi ràng buộc chính là filtered retrieval và vận hành gọn; Weaviate phù hợp khi cần kết hợp keyword với semantic search; Milvus phù hợp khi scale và HA quan trọng hơn độ đơn giản triển khai.

5 Đánh giá so sánh

5.1 Thiết lập và phương pháp

Nhóm benchmark ba database trong cùng một pipeline để giảm sai lệch do môi trường. Kết quả dưới đây là **snapshot** trên corpus synthetic 10K chunks và 200 queries; vì vậy nó phù hợp để so sánh trong điều kiện kiểm soát, không phải ranking chung cho mọi workload production.

Bảng 4: Cấu hình thực nghiệm và fairness protocol.

Thành phần	Chi tiết
Hardware	AMD Ryzen 7 5800H, 16 logical CPUs, 14 GiB RAM, NVMe SSD 512 GB
Runtime	Arch Linux, Docker Compose v2, Ollama local
Embedding / LLM	nomic-embed-text 768 chiều; qwen2.5:1.5b cho RAG chat
Index params	HNSW $M = 16$, $ef_construction = 128$, $ef_search = 64$, Cosine distance
Dataset	10.000 synthetic chunks, seed = 42; 200 golden queries có tag ground-truth [CID:xxxx]
Metrics	Recall@1/5/10, MRR, average latency, recall-latency tradeoff, RAM footprint, DX score

Recall@K đo tỷ lệ ground-truth xuất hiện trong top K kết quả, còn MRR đo vị trí trung bình của kết quả đúng:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}.$$

Toàn bộ tham số HNSW được đọc từ cấu hình chung thay vì hardcode riêng cho từng wrapper, giúp so sánh nhất quán hơn.

5.2 Kết quả snapshot

Bảng 5: Recall@K, MRR và latency trung bình (corpus = 10K, queries = 200).

Engine	Recall@1	Recall@5	Recall@10	MRR	Avg Latency (ms)	Errors
Milvus	18.0	34.0	44.0	0.2492	4.14	0
Qdrant	3.0	7.5	9.5	0.0467	4.83	0
Weaviate	3.0	8.0	9.5	0.0472	14.47	0

Trong snapshot này, Milvus đạt Recall@K và MRR cao nhất, đồng thời giữ average latency thấp. Qdrant và Weaviate có Recall@10 tương đương nhau, nhưng Qdrant nhanh hơn trong dense retrieval. Tuy nhiên, cách đọc đúng không phải là “Milvus luôn tốt nhất”: benchmark này dùng synthetic corpus, single-node deployment và một bộ tham số index cố định.

Recall tuyệt đối thấp chủ yếu đến từ *vector collapse*. Corpus synthetic được sinh từ số lượng template ít; nhiều chunk gần như đồng nghĩa nên embedding model gom chúng vào cùng vùng không gian. Khi đó database có thể tìm đúng cụm ngữ nghĩa nhưng chưa chắc trả đúng ID ground-truth cụ thể. Đây là giới hạn của dataset benchmark, không phải bằng chứng rằng các database có chất lượng retrieval thấp trong mọi corpus thật.

5.3 Recall-latency tradeoff và filter

Bảng 6: Tradeoff sweep: Recall (%) / Avg latency (ms) theo top_k.

Engine	K=1	K=2	K=5	K=10	K=20	K=50
Milvus	18.0 / 3.65	24.0 / 3.68	34.0 / 3.67	44.0 / 3.88	62.0 / 3.85	80.0 / 4.17
Qdrant	3.0 / 4.57	3.5 / 4.71	7.5 / 4.80	9.5 / 4.87	17.5 / 5.10	27.0 / 5.82
Weaviate	3.0 / 16.19	3.5 / 15.77	8.0 / 15.39	9.5 / 15.49	15.5 / 14.96	24.5 / 15.66

Milvus giữ đường tradeoff tốt nhất trên snapshot này: Recall tăng đến 80% ở K=50 trong khi latency vẫn quanh 4 ms. Qdrant có latency tăng chậm khi K tăng, cho thấy chi phí biên để lấy thêm kết quả khá nhỏ. Weaviate có latency ổn định quanh 15–16 ms; kết quả này phản ánh dense retrieval path trong benchmark, chưa thể hiện đầy đủ lợi thế native hybrid search của Weaviate.

Qdrant được đo riêng với filter metadata sau warmup: dense-only baseline đạt 3.97 ms; equality filter đạt 5.29 ms; range filter đạt 5.20 ms; combined filter đạt 5.73 ms. Overhead +1.2–1.8 ms cho thấy trong thí nghiệm này payload filter tạo chi phí nhỏ và vẫn phù hợp với RAG cần tenant/ACL filtering.

5.4 Tài nguyên và Developer Experience

Bảng 7: Tài nguyên và DX trong codebase benchmark của nhóm.

Metric	Qdrant	Weaviate	Milvus
RAM sau benchmark	~178 MB	~200–300 MB	~398 MB
Containers cần thiết	1	1	3+
SLOC wrapper	232	306	379
Cyclomatic complexity	24	59	36
Third-party imports	2	4	3
Composite DX score (thấp hơn tốt hơn)	190.0	307.0	290.5

Qdrant có footprint và DX score tốt nhất trong codebase của nhóm, phù hợp với prototype hoặc team nhỏ. Milvus dùng nhiều tài nguyên và dependency hơn, nhưng đó là chi phí đi kèm kiến trúc phân tán. Weaviate nằm giữa hai cực: triển khai vẫn gọn, nhưng wrapper phức tạp hơn vì schema, module và hybrid query surface.

5.5 Hạn chế

Thí nghiệm có bốn giới hạn chính. Thứ nhất, synthetic corpus gây vector collapse nên recall tuyệt đối không đại diện cho tài liệu thật. Thứ hai, benchmark chạy single-node nên chưa đo được lợi thế phân tán của Milvus. Thứ ba, Docker resource limits và warmup/cache có thể ảnh hưởng latency tail. Thứ tư, Weaviate được đánh giá chủ yếu bằng dense retrieval snapshot, trong khi điểm mạnh thiết kế của nó là hybrid retrieval. Vì vậy, kết quả nên được đọc như bằng chứng cho một workload cụ thể, không phải kết luận vĩnh viễn.

6 Thảo luận

Kết quả kiến trúc và benchmark cho thấy lựa chọn Vector Database nên bắt đầu từ ràng buộc hệ thống thay vì từ một bảng xếp hạng tuyệt đối. Ba công cụ đều có thể dùng cho RAG, nhưng mỗi công cụ tối ưu cho một điểm khác nhau: Qdrant cho filtered retrieval gọn nhẹ, Weaviate cho hybrid search, Milvus cho scale và HA.

Bảng 8: Ma trận quyết định theo use case.

Use case	Khuyến nghị
Prototype / POC nhanh	Qdrant, vì triển khai 1 container, wrapper ít dòng code và DX score thấp nhất trong codebase nhóm.
RAG cần keyword + semantic	Weaviate, vì native hybrid search kết hợp BM25 và vector trong cùng query surface [19].
Enterprise hoặc corpus rất lớn	Milvus, vì kiến trúc disaggregated hỗ trợ scale per-role, HA và Knowhere/GPU/DiskANN [9].
Multi-tenant, ACL, filter-heavy RAG	Qdrant, vì payload-oriented design và filter-aware search path phù hợp metadata filtering.
Team nhỏ, ít kinh nghiệm vận hành	Qdrant hoặc Weaviate; Milvus chỉ nên chọn khi nhu cầu scale đủ lớn để bù chi phí ops.

Nếu ưu tiên latency và vận hành gọn, Qdrant là lựa chọn để bắt đầu nhất. Nếu chất lượng retrieval phụ thuộc cả từ khóa chính xác lẫn ngữ nghĩa, Weaviate có lợi thế thiết kế dù dense-only benchmark không phải điểm mạnh nhất của nó. Nếu bài toán cần corpus rất lớn, replica, GPU hoặc tách riêng ingest/search/indexing, Milvus có kiến trúc phù hợp hơn, nhưng yêu cầu đội vận hành hiểu Docker/Kubernetes và các dependency như object storage, etcd, message queue.

Với Qdrant, điểm mạnh không chỉ nằm ở thời gian truy vấn trong snapshot mà còn ở đường cong vận hành. Mô hình collection/point/payload dễ ánh xạ sang ứng dụng RAG thông thường: mỗi chunk có embedding, metadata và nội dung tham chiếu. Khi hệ thống cần lọc theo tenant, nguồn dữ liệu, vai trò người dùng hoặc thời gian cập nhật, payload index giúp giảm phần không gian tìm kiếm trước hoặc trong quá trình truy vấn tùy kế hoạch thực thi. Trong nhóm ba công cụ, Qdrant vì vậy là lựa chọn hợp lý khi team muốn đi từ prototype lên production nhỏ-vừa mà chưa muốn gánh nhiều service phụ trợ. Trade-off là khi yêu cầu scale ngang phức tạp, nhiều pipeline ingest song song hoặc HA nghiêm ngặt, Qdrant cần được thiết kế cụm cẩn thận hơn thay vì chỉ dựa vào cấu hình single-node.

Với Weaviate, cần đọc benchmark dense-only một cách công bằng. Nếu chỉ đo vector search trên cùng embedding, Weaviate có thể không luôn dẫn đầu về latency hoặc footprint; tuy nhiên đó chưa phải workload đại diện nhất cho công cụ này. Weaviate mạnh khi truy vấn gồm cả ngữ nghĩa và từ khóa: ví dụ tài liệu pháp lý có số điều khoản, catalog có mã sản phẩm, ticket hỗ trợ có tên lỗi hoặc log code. Trong các trường hợp này, BM25/inverted index và vector HNSW bổ sung cho nhau, còn hybrid score giúp hệ thống tránh bỏ sót kết quả chứa token chính xác nhưng embedding không đủ

gần. Vì vậy, Weaviate phù hợp với đội phát triển muốn một API retrieval giàu tính năng hơn là một ANN service tối giản.

Với Milvus, lợi thế xuất hiện rõ hơn khi corpus và tổ chức vận hành đủ lớn. Kiến trúc tách Proxy, Coordinator, QueryNode, DataNode và IndexNode cho phép scale từng vai trò theo bottleneck: nhiều truy vấn thì tăng QueryNode, ingest nặng thì chú ý DataNode và message queue, build index lớn thì tăng IndexNode hoặc cân nhắc GPU. Đây là kiểu thiết kế quen thuộc trong Big Data system: đổi sự đơn giản của single binary lấy khả năng mở rộng, cô lập lỗi và tối ưu theo workload. Do đó, kết luận “Milvus đạt Recall@K cao nhất trong snapshot” nên được hiểu là tín hiệu về năng lực index/search, không đồng nghĩa Milvus luôn là lựa chọn tốt nhất cho mọi team.

Một cách đọc thực dụng hơn là xem benchmark như bài kiểm tra điều kiện. Snapshot 10K synthetic chunks và 200 queries giúp so sánh hành vi tương đối trong môi trường kiểm soát, nhưng chưa bao phủ dữ liệu đa miền, update/delete liên tục, phân quyền phức tạp, warm-up cache dài hạn hoặc failure recovery. Khi áp dụng vào dự án thật, nhóm nên tái chạy quy trình đánh giá trên dữ liệu nội bộ, giữ nguyên ba nhóm chỉ số: chất lượng truy hồi, chi phí tài nguyên và độ phức tạp vận hành. Lựa chọn tốt nhất không phải công cụ thắng nhiều cột nhất, mà là công cụ có điểm mạnh trùng với ràng buộc quan trọng nhất của hệ thống.

Xu hướng chung của Vector Database là tiến tới retrieval đa tín hiệu: dense vector, sparse/keyword, metadata filter, reranking và multi-modal embedding. Vì vậy, benchmark trong báo cáo nên được xem là quy trình đánh giá có thể lặp lại, không phải kết luận cuối cùng cho mọi phiên bản và workload.

7 Kết luận

Báo cáo đã phân tích Qdrant, Weaviate và Milvus từ bốn góc nhìn: mục tiêu thiết kế, trạng thái mã nguồn mở/chi phí, kiến trúc nội bộ và kết quả benchmark trong hệ thống RAG của nhóm. Cả ba đều là lựa chọn nghiêm túc cho vector search, nhưng chúng tối ưu cho ba ràng buộc khác nhau.

Trong snapshot 10K synthetic chunks, Milvus đạt Recall@K cao nhất; Qdrant nổi bật ở footprint, latency filter và DX; Weaviate nổi bật ở native hybrid retrieval. Do đó, kết luận quan trọng không phải là chọn một “winner” tuyệt đối, mà là chọn công cụ theo use case: Qdrant cho filtered RAG gọn nhẹ, Weaviate cho keyword + semantic retrieval, và Milvus cho scale/HA.

Đóng góp chính của nhóm là xây dựng một quy trình đánh giá có thể lặp lại: cùng corpus, cùng query, cùng tham số index, cùng pipeline và cùng dashboard trực quan hóa. Đây là cách tiếp cận thực tế hơn so với việc so sánh công cụ chỉ bằng mô tả marketing hoặc một metric đơn lẻ.

Tài liệu

- [1] ANN Benchmarks. ANN benchmarks — benchmarking nearest neighbors. <https://ann-benchmarks.com/>, 2024. Accessed: 2026-05-24.
- [2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.
- [3] DB-Engines. DB-Engines ranking of vector DBMS. <https://db-engines.com/en/ranking/vector+dbms>, 2026. Accessed: 2026-05-24.
- [4] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [6] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [7] Milvus. DiskANN index — Milvus documentation. https://milvus.io/docs/disk_index.md, 2024. Accessed: 2026-05-24.
- [8] Milvus. Knowhere — Milvus vector search engine. <https://milvus.io/docs/knowhere.md>, 2024. Accessed: 2026-05-24.
- [9] Milvus. Milvus architecture overview. https://milvus.io/docs/architecture_overview.md, 2024. Accessed: 2026-05-24.
- [10] Milvus. Milvus documentation. <https://milvus.io/docs>, 2024. Accessed: 2026-05-24.

- [11] Qdrant. Filtrable HNSW — Qdrant articles. <https://qdrant.tech/articles/filtrable-hnsw/>, 2023. Accessed: 2026-05-24.
- [12] Qdrant. Qdrant documentation. <https://qdrant.tech/documentation/>, 2024. Accessed: 2026-05-24.
- [13] Qdrant. Qdrant storage architecture. <https://qdrant.tech/documentation/concepts/storage/>, 2024. Accessed: 2026-05-24.
- [14] Qdrant. Qdrant cloud pricing. <https://qdrant.tech/pricing/>, 2026. Accessed: 2026-06-07.
- [15] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [16] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, 2019.
- [17] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 2614–2627, 2021.
- [18] Weaviate. Weaviate documentation. <https://weaviate.io/developers/weaviate>, 2024. Accessed: 2026-05-24.
- [19] Weaviate. Weaviate hybrid search. <https://weaviate.io/developers/weaviate/search/hybrid>, 2024. Accessed: 2026-05-24.
- [20] Weaviate. Weaviate modules. <https://weaviate.io/developers/weaviate/modules>, 2024. Accessed: 2026-05-24.

- [21] Weaviate. Weaviate pricing. <https://weaviate.io/pricing.html>, 2026. Accessed: 2026-06-07.
- [22] Zilliz. Try zilliz cloud for free. <https://docs.zilliz.com/docs/free-trials>, 2026. Accessed: 2026-06-07.